# CSCE 46103/56103 Artificial Intelligence – Fall 2026

# Multilayer Perceptrons

Jungtaek Kim
jungtaek.kim@uark.edu

University of Arkansas

November 3, 2026

UNIVERSITY OF ARKANSAS

# This Material's Topics

- Neurons, activation functions, and hidden layers.
- Computing predictions with a feedforward network.
- Encoding inputs and choosing outputs for a learning task.

The Central Question

How do connected, trainable neurons turn an input into a prediction?

Feedforward networks and activation functions: Russell and Norvig [2020, Sec. 21.1].

# From Linear Predictors to Neurons

Recall the linear predictor $h_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w}^\mathsf{T}\mathbf{x} + b$. An artificial neuron applies an activation function $g$ to this weighted sum:

$$z = b + \sum_{j=1}^{d} w_j x_j, \qquad a = g(z).$$

- The weights $w_j$ control the contribution of each input.
- The bias $b$ shifts the input to the activation function.
- The activation $a$ becomes an input to another neuron or an output.

Feedforward networks and activation functions: Russell and Norvig [2020, Sec. 21.1].

# From Linear Predictors to Neurons

Recall the linear predictor $h_{\mathbf{w},b}(\mathbf{x}) = \mathbf{w}^{\mathsf{T}}\mathbf{x} + b$. An artificial neuron applies an activation function $g$ to this weighted sum:

$$z = b + \sum_{j=1}^{d} w_j x_j, \qquad a = g(z).$$

- The weights $w_j$ control the contribution of each input.
- The bias $b$ shifts the input to the activation function.
- The activation $a$ becomes an input to another neuron or an output.

## The Same Learning Framework

The network defines $h_{\mathbf{w}}(\mathbf{x})$. Training chooses its weights and biases to reduce a loss on labeled examples.

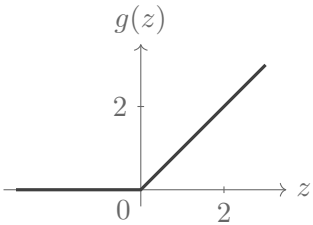
Feedforward networks and activation functions: Russell and Norvig [2020, Sec. 21.1].

UNIVERSITY OF ARKANSAS

# Activation Functions

Rectified Linear Unit (ReLU)

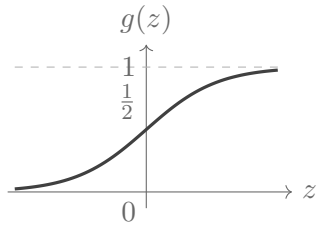
$$g(z) = \max(0, z).$$




Zero for $z < 0$; equal to $z$ for $z \geq 0$.

Logistic Sigmoid

$$\sigma(z) = \frac{1}{1 + e^{-z}}.$$




Maps any real input into $(0, 1)$.

Another activation, $\tanh(z)$, maps real inputs into $(-1, 1)$.

# Nonlinear Activations

If every layer is linear, composing two layers gives another linear map:

$$W_2(W_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2 = (W_2W_1)\mathbf{x} + (W_2\mathbf{b}_1 + \mathbf{b}_2).$$

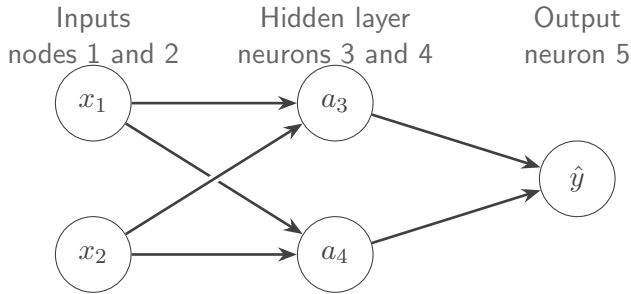
| $x_1$ | $x_2$ | XOR |
|---|---|---|
| 0 | 0 | 0 |
| 0 | 1 | 1 |
| 1 | 0 | 1 |
| 1 | 1 | 0 |

- A single straight decision boundary cannot separate the XOR classes.
- Nonlinear hidden neurons allow the network to combine simpler regions into a more expressive prediction rule.

Linear separability and multilayer networks: Russell and Norvig [2020, Sec. 21.1].

UNIVERSITY OF ARKANSAS

# A Feedforward Network




Each hidden or output neuron includes a bias.

- Information flows from inputs through the hidden layer to the output.
- Each connection has a weight; each hidden or output neuron has a bias.
- Hidden activations are computed values, not supplied target labels.

Two-input, two-hidden-neuron architecture adapted from Russell and Norvig [2020, Fig. 21.3].

# A Small Network's Parameters

Let $w_{i,j}$ be the weight from node $i$ to neuron $j$; $w_{0,j}$ is neuron $j$'s bias.

$$a_3 = \mathrm{ReLU}(w_{0,3} + w_{1,3}x_1 + w_{2,3}x_2),$$
$$a_4 = \mathrm{ReLU}(w_{0,4} + w_{1,4}x_1 + w_{2,4}x_2),$$
$$\hat{y} = w_{0,5} + w_{3,5}a_3 + w_{4,5}a_4.$$

| Neuron | First incoming weight | Second incoming weight | Bias |
|---|---|---|---|
| Hidden 3 | $w_{1,3} = 1$ | $w_{2,3} = 0$ | $w_{0,3} = 0$ |
| Hidden 4 | $w_{1,4} = 0$ | $w_{2,4} = 1$ | $w_{0,4} = 0$ |
| Output 5 | $w_{3,5} = 0.5$ | $w_{4,5} = 0.5$ | $w_{0,5} = 0$ |

Architecture from Russell and Norvig [2020, Fig. 21.3]; numerical values are a classroom example.

# A Forward Pass

Use the preceding weights with input $\mathbf{x} = (1, 2)^{\mathsf{T}}$ and target $y = 2$.

$$a_3 = \mathrm{ReLU}(0 + 1 \cdot 1 + 0 \cdot 2) = 1,$$
$$a_4 = \mathrm{ReLU}(0 + 0 \cdot 1 + 1 \cdot 2) = 2,$$
$$\hat{y} = 0 + 0.5 \cdot 1 + 0.5 \cdot 2 = 1.5.$$

Architecture from Russell and Norvig [2020, Fig. 21.3]; numerical values are a classroom example.

# A Forward Pass

Use the preceding weights with input $\mathbf{x} = (1, 2)^\mathsf{T}$ and target $y = 2$.

$$a_3 = \mathrm{ReLU}(0 + 1 \cdot 1 + 0 \cdot 2) = 1,$$
$$a_4 = \mathrm{ReLU}(0 + 0 \cdot 1 + 1 \cdot 2) = 2,$$
$$\hat{y} = 0 + 0.5 \cdot 1 + 0.5 \cdot 2 = 1.5.$$

With squared loss,

$$L(\hat{y}, y) = (\hat{y} - y)^2 = (1.5 - 2)^2 = 0.25.$$

## Prediction, Then Comparison

The forward pass computes $\hat{y}$ from $\mathbf{x}$. The target $y$ is used to compute the loss during training.

Architecture from Russell and Norvig [2020, Fig. 21.3]; numerical values are a classroom example.

# Encoding the Inputs

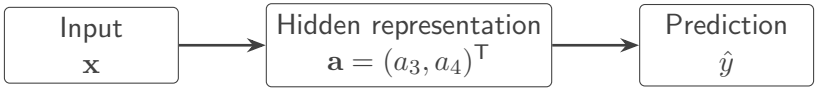
A neural network receives numerical inputs. A categorical attribute can use a one-hot encoding:

| Restaurant Type | French | Italian | Thai | Burger |
|---|---|---|---|---|
| French | 1 | 0 | 0 | 0 |
| Italian | 0 | 1 | 0 | 0 |
| Thai | 0 | 0 | 1 | 0 |
| Burger | 0 | 0 | 0 | 1 |

- Each coordinate indicates membership in one category.
- One categorical attribute becomes four numerical inputs.
- Numerical attributes can be included as additional coordinates.

Input encodings: Russell and Norvig [2020, Sec. 21.2.1].

# Hidden Representations




- A hidden neuron responds to a weighted combination of its inputs.
- Multiple neurons provide multiple features for the output layer.
- Additional layers compose these features into new representations.

Hidden representations: Russell and Norvig [2020, Sec. 21.2.3].

# Outputs and Loss Functions

Let $z$ denote the final weighted sum before the output activation.

| Task | Output | Common loss |
|---|---|---|
| Regression | $\hat{y} = z \in \mathbb{R}$ | Squared loss $(\hat{y} - y)^2$ |
| Binary classification | $p = \sigma(z)$ for class 1 | $-y \log p - (1 - y) \log(1 - p)$ |
| Multiclass classification | Softmax probabilities $p_1, \ldots, p_K$ | $-\log p_y$ for the correct class $y$ |

## Cross-Entropy

For classification, these losses decrease as the model assigns more probability to the correct class. Binary targets are $y \in \{0, 1\}$.

Outputs and loss functions: Russell and Norvig [2020, Sec. 21.2.2].

# Softmax Probabilities

For $K$ output scores, or logits, $z_1, \ldots, z_K$,

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad p_k > 0, \qquad \sum_{k=1}^{K} p_k = 1.$$

| **Class** | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Logit $z_k$ | 5 | 2 | 0 | $-2$ |
| Probability $p_k$ | 0.9457 | 0.0471 | 0.0064 | 0.0009 |

Softmax example from Russell and Norvig [2020, Sec. 21.2.2]. Displayed probabilities are rounded.

# Softmax Probabilities

For $K$ output scores, or logits, $z_1, \ldots, z_K$,

$$p_k = \frac{e^{z_k}}{\sum_{j=1}^{K} e^{z_j}}, \qquad p_k > 0, \qquad \sum_{k=1}^{K} p_k = 1.$$

| **Class** | 1 | 2 | 3 | 4 |
|---|---|---|---|---|
| Logit $z_k$ | 5 | 2 | 0 | $-2$ |
| Probability $p_k$ | 0.9457 | 0.0471 | 0.0064 | 0.0009 |

- The largest score gives the largest probability: class 1 is predicted.
- The scores compete through their shared denominator.

Softmax example from Russell and Norvig [2020, Sec. 21.2.2]. Displayed probabilities are rounded.

# Checkpoint: Computing with the Network

Keep the weights of our small network, but use input $\mathbf{x} = (2, 1)^{\mathsf{T}}$:

$$a_3 = \mathrm{ReLU}(x_1), \qquad a_4 = \mathrm{ReLU}(x_2), \qquad \hat{y} = 0.5a_3 + 0.5a_4.$$

1. What are $a_3$, $a_4$, and $\hat{y}$?
2. If $y = 2$, what is the squared loss?

Architecture from Russell and Norvig [2020, Fig. 21.3]; numerical values are a classroom example.

# Checkpoint: Computing with the Network

Keep the weights of our small network, but use input $\mathbf{x} = (2, 1)^{\mathsf{T}}$:

$$a_3 = \mathrm{ReLU}(x_1), \qquad a_4 = \mathrm{ReLU}(x_2), \qquad \hat{y} = 0.5a_3 + 0.5a_4.$$

1. What are $a_3$, $a_4$, and $\hat{y}$?
2. If $y = 2$, what is the squared loss?

Answers

$a_3 = 2$, $a_4 = 1$, and $\hat{y} = 1.5$; the loss is $0.25$.

Architecture from Russell and Norvig [2020, Fig. 21.3]; numerical values are a classroom example.

# Main Takeaways

- A neuron applies an activation function to a weighted sum and bias.
- Nonlinear hidden layers give networks expressive representations.
- A forward pass computes hidden activations and the output in order.
- Input encodings, output activations, and losses depend on the task.

Russell and Norvig [2020, Secs. 21.1–21.2].

# Thank You

Any questions?

# References

S. J. Russell and P. Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4th edition, 2020. URL `https://aima.cs.berkeley.edu/`.